RE033 - Mobile App Security and Data Protection Standards

Research Paper for Food Remedy API T1/2026

Prepared by: Ocean Ocean

Date: May 2026

Priority: Medium | Category: Research | Level: 3

Abstract

Mobile health and nutrition applications collect sensitive personal information such as food intake, allergies, symptoms, medication notes, activity data and account identifiers. Because this information can reveal details about a user's health, habits and lifestyle, it must be protected with strong security and privacy controls. This paper examines the main security practices required for the Food Remedy API mobile application, with a focus on authentication, access control, encryption, secure storage, API security, platform-specific protection for iOS and Android, and privacy compliance. The research draws on recognised security guidance including OWASP MASVS, OWASP API Security Top 10, HHS guidance for mobile health app developers, FTC best practices for health apps, NIST digital identity guidance, GDPR principles, and official Apple and Android secure storage documentation. The paper concludes with practical recommendations that the Food Remedy API team can apply during design, development, testing and release.

1. Introduction

The Food Remedy API project is a health and nutrition-related application environment where users may record or receive information connected to food, allergies, symptoms, nutrition preferences and other personal details. A mobile application connected to this type of API must be designed with security and privacy from the beginning, not added only after development. If user data is leaked, modified or accessed by the wrong person, the impact can include embarrassment, loss of trust, identity misuse, discrimination or unsafe health-related decisions.

The purpose of RE033 is to research mobile app security and data protection standards that are suitable for an application handling user data in a health and nutrition context. The research focuses on four main questions: how users should authenticate securely, how sensitive information should be stored and encrypted, how APIs should be protected, and how common mobile vulnerabilities can be reduced through recognised standards and testing practices.

This paper is written as a practical research document for the Food Remedy API team. It does not replace legal advice, but it provides a clear technical and privacy-focused foundation for making safer design and development decisions.

2. Security Context for Health and Nutrition Apps

Health and nutrition apps are different from ordinary lifestyle apps because the information they collect can reveal sensitive details about a person's body, habits, conditions and needs. Even simple data such as allergies, dietary restrictions, weight goals, medication reminders or symptom tracking can become

sensitive when linked to an identifiable user account. For this reason, the app should apply data minimisation, meaning it should collect only the information required for the feature being provided.

The HHS guidance for mobile health app developers explains that privacy and security protections can increase the value of technology products by giving users confidence that information is secure and used only as expected. The FTC also recommends that mobile health app developers minimise data, limit access and permissions, consider authentication, and design security into the product from the start. These recommendations are directly relevant to the Food Remedy API because the system may process personal and health-related information through mobile clients and backend APIs.

The GDPR also treats data concerning health as a special category of personal data, meaning it requires a stronger privacy basis and careful handling. Even where a project is not directly regulated by GDPR or HIPAA, these frameworks provide useful principles for responsible design: collect less data, explain how data is used, restrict access, protect stored information and give users control over their data where possible.

3. Secure Authentication and Access Control

Authentication is the first major security layer because it confirms that the person using the app is the legitimate user. For Food Remedy API, the application should not rely only on simple username and password login. Passwords should be strong, checked against common or compromised password patterns where possible, and never stored in plain text. On the server, passwords should be salted and hashed using modern password hashing methods such as Argon2, bcrypt or scrypt.

Multi-factor authentication should be considered for sensitive actions such as exporting health reports, changing an email address, deleting an account or sharing information with another person. MFA reduces the risk that a stolen password alone can give an attacker full access. Biometric unlock, such as fingerprint or face recognition, can improve user convenience on mobile devices, but it should be used as a local re-authentication mechanism supported by secure server-side session controls.

For mobile login flows, OAuth 2.0 with PKCE is recommended because it is designed for public clients such as mobile apps. Access tokens should be short-lived, refresh tokens should be protected, and all tokens must be stored only in platform-backed secure storage. The app should also use session timeouts and re-authentication for high-risk actions.

Access control is equally important. A user should only be able to access their own profile, meal records, allergy information and saved preferences. If the project later includes clinicians, coaches or administrators, the backend should enforce role-based access control. However, role checks alone are not enough. Every API endpoint must also perform object-level authorisation checks so that one user cannot access another user’s record by changing an ID in a request URL or body.

4. Encryption and Secure Storage

Data protection must cover both data in transit and data at rest. Data in transit is information moving between the mobile app and the backend API. The Food Remedy API should enforce HTTPS for every request and should reject plain HTTP. Modern TLS should be used so that attackers on public Wi-Fi cannot read or modify user data. For higher-risk endpoints, certificate pinning may also be considered, although it must be managed carefully because certificate rotation can break app connectivity if implemented poorly.

Data at rest is information stored on the mobile device, server database, backups or logs. On mobile devices, sensitive data should not be stored in plain text files, external storage, screenshots, local logs or analytics tools. Small secrets such as access tokens, refresh tokens and cryptographic keys should be

stored using iOS Keychain or Android Keystore-backed storage. Apple describes Keychain as encrypted storage for small pieces of user data, and Android Keystore is designed to keep cryptographic keys harder to extract from the device.

The server side should also encrypt sensitive database fields and backups. Database access should be limited to service accounts and authorised administrators only. Logs should be carefully designed so they do not accidentally store health information, authentication tokens, full request bodies or personal identifiers. Crash reporting and analytics tools should also be configured to avoid collecting sensitive content.

5. API and Backend Security

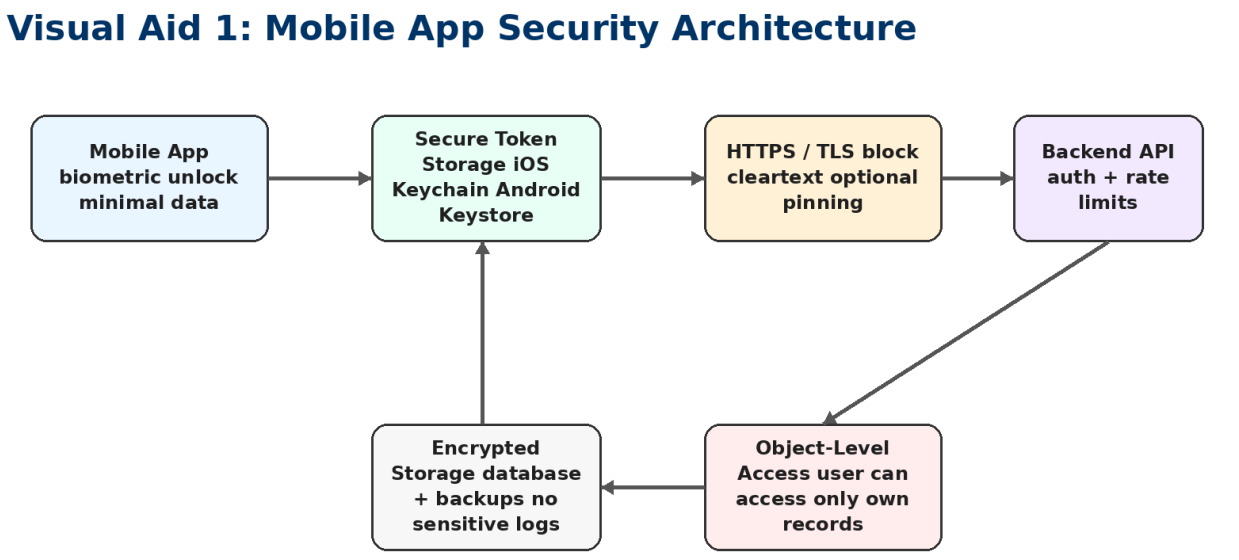

Figure 1. Security architecture showing mobile, transport, API, authorisation and storage controls.

The backend API is one of the most important attack surfaces for a mobile health and nutrition app. OWASP API Security Top 10 identifies broken object-level authorisation as a leading API risk because attackers can manipulate object IDs in API requests to access data they do not own. This risk is especially important for Food Remedy API because records such as meal logs, allergy profiles or saved reports may be retrieved using database identifiers.

All API endpoints that return or modify user data should require authentication. The backend should verify the token, identify the user, check the user's role, and confirm that the requested object belongs to that user or is otherwise permitted. The API should not trust the mobile app interface to hide buttons or screens; security decisions must be enforced on the server.

API rate limiting should be added to reduce brute-force login attempts, credential stuffing, scraping and denial-of-service style abuse. Input validation should be applied to prevent injection attacks, malformed payloads and unsafe file uploads. Error messages should be useful for debugging during development but should not expose stack traces, database names, tokens or internal system details in production.

The team should maintain an API inventory so that old, test or debug endpoints are not accidentally left online. Any third-party services used for nutrition data, analytics, push notifications or authentication should be reviewed for privacy and security impact before integration.

6. Platform-Specific Security for iOS and Android

Mobile security also depends on using the built-in protection features of each platform. On iOS, the app should use Keychain for tokens and secrets, Data Protection classes for files, App Transport Security for secure network communication, and Face ID or Touch ID for local re-authentication where appropriate. Sensitive information should not appear in push notification previews or app switcher screenshots.

On Android, the app should use Android Keystore for cryptographic keys and encrypted storage methods for tokens and sensitive cached data. Developers should avoid external storage for private health information, limit exported activities and services, enforce permissions on components, and use Network Security Configuration to block cleartext traffic. The app should also be tested on different Android versions because device fragmentation can create inconsistent security behaviour.

7. Common Vulnerabilities and Mitigation Strategies

The most common vulnerabilities for a mobile health and nutrition app include weak authentication, insecure local storage, unencrypted traffic, broken access control, insecure APIs, excessive permissions, outdated libraries and poor privacy practices. These problems are preventable when security is included in the design and testing process.

Weak authentication can be reduced through strong password rules, secure password hashing, MFA for sensitive actions, login attempt limits and session expiry. Insecure storage can be reduced by using Keychain, Keystore-backed storage and encrypted databases, while avoiding logs and screenshots that contain sensitive data. Broken access control can be reduced by enforcing object-level checks on every backend endpoint. Unencrypted traffic can be prevented by requiring HTTPS and blocking cleartext connections. Outdated libraries can be managed through dependency scanning and regular updates.

Poor privacy practices are also a security risk. The app should not collect unnecessary data, should explain clearly what information is collected and why, and should give users reasonable control over their information. If analytics or third-party SDKs are used, they should be configured to avoid collecting sensitive health or nutrition data unless there is a clear reason and proper consent.

8. Practical Security Checklist for Food Remedy API

Visual Aid 2: Data Protection Layers for Food Remedy API

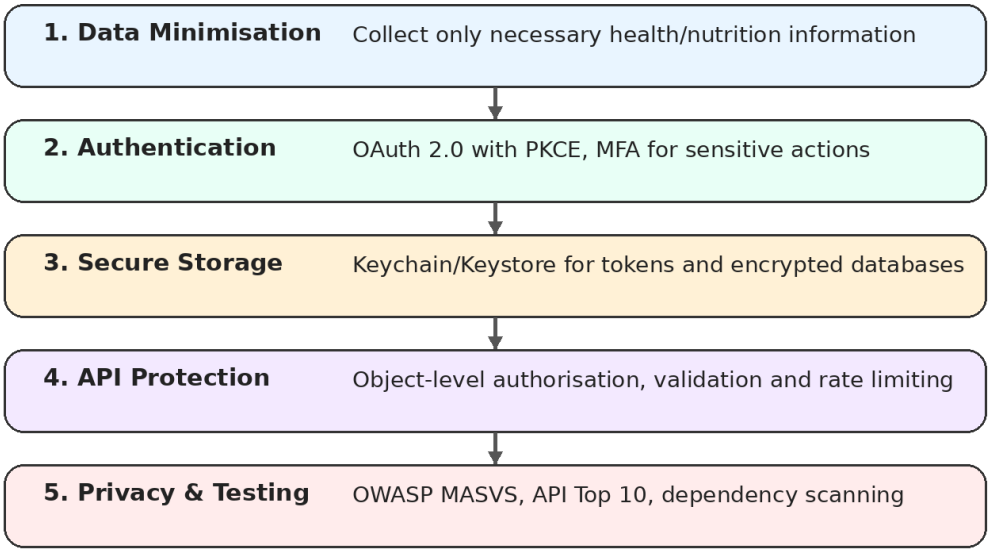

Figure 2. Data protection layers that should be applied across the Food Remedy API mobile system.

Area	Required Practice	Reason for Food Remedy API
Authentication	Use strong passwords, salted password hashing, MFA for sensitive actions and secure session management.	Reduces account takeover risk and protects personal nutrition/health records.
Token storage	Store tokens only in iOS Keychain or Android secure storage backed by Keystore.	Prevents easy theft of access tokens from the mobile device.
Transport security	Enforce HTTPS/TLS for all app-to-API communication and block cleartext traffic.	Protects data on public Wi-Fi and other untrusted networks.
Access control	Apply RBAC where needed and object-level authorisation on every API request.	Prevents users from viewing or changing records that do not belong to them.
Local data	Avoid plain text files, logs, screenshots and external storage for sensitive data.	Reduces harm if a phone is lost, stolen or compromised.
Backend storage	Encrypt sensitive database fields and backups; restrict database and admin access.	Protects user information if server systems are exposed.
API protection	Use rate limiting, input validation, safe error handling and API inventory management.	Reduces brute-force attacks, injection risk and accidental exposure.
Privacy	Collect only necessary data and explain how it is used.	Builds user trust and supports privacy-by-design principles.
Testing	Use OWASP MASVS and API Security Top 10 as testing checklists before release.	Provides a recognised method for finding mobile and API security weaknesses.

9. Recommendations

- Adopt OWASP MASVS as the main mobile security checklist for the Food Remedy mobile application.
- Use OAuth 2.0 with PKCE for mobile authentication and store tokens only in platform secure storage.
- Enforce HTTPS/TLS for all API communication and disable cleartext network traffic in production builds.
- Apply object-level authorisation checks to every endpoint that accesses user-specific data.
- Use encryption for sensitive local data, server databases and backups.
- Avoid storing health, allergy, symptom or nutrition details in logs, analytics payloads or push notification text.
- Create a privacy-by-design data map showing what data is collected, where it is stored, who can access it and why it is needed.
- Perform security testing before release using OWASP MASVS, OWASP API Security Top 10 and dependency vulnerability scanning.
- Prepare an incident response process that explains how the team will detect, investigate and communicate a possible data breach.

10. Conclusion

Mobile app security and data protection are essential for the Food Remedy API because the project may handle personal and health-related nutrition information. The strongest approach is to combine secure authentication, strict access control, encrypted communication, safe storage, backend API protection, privacy-by-design and continuous testing. The app should use iOS and Android platform security features rather than custom storage methods, and the backend should enforce all important security checks instead of relying on the mobile interface alone.

The most important conclusion from this research is that security must be treated as a core feature of the product. By following OWASP MASVS, OWASP API Security Top 10, HHS and FTC health app guidance, NIST authentication guidance and GDPR privacy principles, the Food Remedy API team can reduce the risk of data breaches and build a safer, more trustworthy mobile application.

References

Apple. (2024). Keychain data protection. Apple Platform Security. https://support.apple.com/guide/security/keychain-data-protection-secb0694df1a/web

Apple Developer. (n.d.). Keychain services. https://developer.apple.com/documentation/security/keychain-services

Android Developers. (2026). Android Keystore system. https://developer.android.com/privacy-and-security/keystore

European Union. (2016). General Data Protection Regulation, Article 9: Processing of special categories of personal data. https://gdpr-info.eu/art-9-gdpr/

Federal Trade Commission. (2016). Mobile health app developers: FTC best practices. https://www.ftc.gov/business-guidance/resources/mobile-health-app-developers-ftc-best-practices

National Institute of Standards and Technology. (2020). Digital Identity Guidelines: Authentication and Lifecycle Management (SP 800-63B). https://pages.nist.gov/800-63-3/sp800-63b.html

OWASP Foundation. (2023). OWASP API Security Top 10 - 2023. https://owasp.org/API-Security/editions/2023/en/0x11-t10/

OWASP Foundation. (n.d.). Mobile Application Security Verification Standard (MASVS). https://mas.owasp.org/MASVS/

U.S. Department of Health and Human Services. (2026). Resources for mobile health apps developers. https://www.hhs.gov/hipaa/for-professionals/special-topics/health-apps/index.html